Relatório Técnico de Entrega 8 — Sistema Web de Monitoramento e Controle

Curso: Internet das Coisas III — IFMG Campus Ouro Branco

Professor: Charles Tim Batista Garrocho

Aluno: Daniel

Dispositivo: `esp32_temp` (ESP32 DevKit V1)

Projeto: Sistema de Alerta de Temperatura e Saúde Ambiental IoT

Data: Novembro / 2026

---

1. Resumo Executivo da Entrega

A **Entrega 8** consolida a camada de apresentação e interação com o usuário através do desenvolvimento da **Interface Web (Single Page Application em React 18 + TypeScript + Vite + TailwindCSS)**, integrada em tempo real com a API REST do Gateway Node.js.

O sistema web permite ao operador monitorar continuamente o **estado atual dos sensores** (Temperatura, Umidade Relativa, Nível de Ruído e Luminosidade), acompanhar a **disponibilidade (`Online`/`Offline`) e o horário do último contato (`lastSeen`)**, explorar o **histórico de telemetria** em gráficos temporais e tabelas paginadas com filtro por data e exportação CSV, além de realizar o **acionamento remoto do atuador** com atualização visual sincronizada com o **estado confirmado (`ACK`/`state`) pelo ESP32**.

---

2. Tabela de Conformidade dos Produtos Esperados

#	Produto Esperado	Requisito da Disciplina	Status	Implementação / Evidência
1	**Interface web funcional**	SPA React 18 responsiva em Dark Mode	**100% Atendido**	`frontend/src/App.tsx` (Container `iot-frontend` na porta `3000`)
2	**Visualização do estado atual do sensor**	Exibição de Temperatura, Umidade, Ruído, Lux e Tela ST7789	**100% Atendido**	`MetricCard.tsx`, `ST7789Display.tsx` e Figura da Seção 5.1
3	**Visualização do histórico de telemetria**	Curvas temporais e tabela paginada com filtro de data e CSV	**100% Atendido**	`HistoryChart.tsx`, `HistoryTable.tsx` e Figuras 5.2 e 5.3
4	**Exibição da presença ou último contato**	Indicador `Online`/`Offline` e carimbo de data/hora (`lastSeen`)	**100% Atendido**	`DeviceBadge.tsx` e cabeçalho em `App.tsx` (Figura 5.1)
5	**Controles para acionamento remoto**	Botão interativo com estado de carregamento (`Enviando & Aguardando ACK...`)	**100% Atendido**	`ActuatorControl.tsx` e Figuras da Seção 5.4
6	**Integração entre frontend e API**	Polling assíncrono (`fetch`) autenticado via HTTP Basic Auth	**100% Atendido**	`fetchDeviceList`, `fetchDeviceData` e `fetchHistory` (`App.tsx`)
7	**Envio de comandos pelo backend**	Requisição `POST /api/devices/:id/command` publicada via MQTT	**100% Atendido**	`handleToggleActuator` (`App.tsx`) -> `gatewayMQTT.sendCommand`
8	**Atualização pelo estado confirmado**	UI atualiza estado após retorno `200 OK` com `ACK/state` do ESP32	**100% Atendido**	Sincronização pós-ACK no Redis e recarga imediata na UI
9	**Evidência do monitoramento e controle**	Capturas reais de todas as seções e ciclo completo de comando	**100% Atendido**	Seção 5 deste relatório (5 evidências visuais em tempo real)

3. Arquitetura de Integração Frontend ↔ API REST ↔ MQTT ↔ ESP32

■ Log de Execução / Diagnóstico

```
[Usuário no Navegador Web — http://localhost:3000]
       ■
       ■■■■ 1. Monitoramento Contínuo (Polling a cada 3s com Basic
           Auth):
       ■       • GET /api/devices              ■■■ Lista dispositivos e
           prioriza nó online
       ■       • GET /api/devices/esp32_temp   ■■■ Atualiza Presença
           (DeviceBadge), Sensores (MetricCard) e ST7789
       ■       • GET /api/devices/.../history  ■■■ Atualiza Gráfico
           Temporal (HistoryChart) e Tabela (HistoryTable)
       ■
       ■■■■ 2. Controle Remoto Bidirecional com Confirmação
           (ActuatorControl):
               • Clique no botão "■ Ligar Atuador" / "■ Desligar
                 Atuador"
               • POST /api/devices/esp32_temp/command
                 {"action":"set","target":"led","state":true}
                    ■
                    ▼
               [Backend Gateway Node.js] ■■■ Publica em
                 ifmg/iot3/turmaA/daniel/esp32_temp/command
                    ■
                    ▼
               [ESP32 DevKit V1]         ■■■ Aciona GPIO 12 (LED) e GPIO
                 14 (Relé)
                    ■                    ■■■ Publica confirmação em
                        .../state (retain=true)
                    ▼
               [Backend Gateway Node.js] ■■■ Recebe confirmação,
                 atualiza Redis e responde HTTP 200 OK
                    ■
                    ▼
               [Interface Web React]     ■■■ Recebe confirmação e
                 atualiza ST7789 ("ATUADOR: LIGADO") e botão
```

4. Componentes e Trechos de Código-Fonte do Sistema Web

4.1 Integração com a API REST e Sincronização de Estado (`frontend/src/App.tsx`)

■ TypeScript / Gateway Node.js

```
const API_BASE =
  import.meta.env.VITE_API_URL ||
  (typeof window !== 'undefined' && window.location.hostname &&
      window.location.hostname !== 'localhost'
    ? `http://${window.location.hostname}:5001/api`
    : 'http://localhost:5001/api');
const AUTH_HEADER = 'Basic ' + btoa('admin:admin123');

const fetchDeviceData = async (targetId = selectedDeviceId) => {
  try {
    const res = await fetch(`${API_BASE}/devices/${targetId}`, {
      headers: { Authorization: AUTH_HEADER },
    });
    if (res.ok) {
      const data = await res.json();
      setDevice(data);
    }
  } catch (err) {
    console.error('Erro ao buscar dispositivo:', err);
  }
};

const fetchHistory = async (targetId = selectedDeviceId, dateFilter =
    selectedDate) => {
  try {
    const url =
        `${API_BASE}/devices/${targetId}/history?limit=2000${dateFilter
        ? `&date=${dateFilter}` : ''}`;
    const res = await fetch(url, {
      headers: { Authorization: AUTH_HEADER },
    });
    if (res.ok) {
      const data = await res.json();
      setHistory(data);
    }
  } catch (err) {
    console.error('Erro ao buscar histórico:', err);
  }
};
```

4.2 Envio de Comando Remoto e Atualização Após Confirmação do ESP32 (`frontend/src/App.tsx`)

■ TypeScript / Gateway Node.js

```
const handleToggleActuator = async () => {
  if (!device) return;
  const currentState = device.state?.actuatorState || false;
  const newState = !currentState;

  setIsLoadingCommand(true);
  try {
    const res = await
        fetch(`${API_BASE}/devices/${selectedDeviceId}/command`, {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json',
        Authorization: AUTH_HEADER,
      },
      body: JSON.stringify({ action: 'set', target: 'led', state:
          newState, value: newState }),
    });

    const result = await res.json();
    if (res.ok && result.success) {
      // Atualiza a interface imediatamente com o estado confirmado pelo
          ESP32 no Redis
      alert(`■ Comando confirmado com sucesso pelo ESP32 (ACK/State)!
          Atuador: ${newState ? 'LIGADO' : 'DESLIGADO'}`);
      fetchDeviceData();
    } else {
      alert(`■ Falha no envio: ${result.message || 'Timeout de resposta
          ACK do dispositivo.'}`);
    }
  } catch {
    alert('■ Erro de conexão com o Backend.');
  } finally {
    setIsLoadingCommand(false);
  }
};
```

4.3 Componente de Presença e Último Contato (`frontend/src/components/DeviceBadge.tsx`)

■ TypeScript / Gateway Node.js

```
export const DeviceBadge: React.FC<DeviceBadgeProps> = ({ status,
    lastSeen }) => {
  const isOnline = status === 'online';
  const formattedTime = lastSeen ? new
      Date(lastSeen).toLocaleTimeString() : '--:--';

  return (
    <div
      className={`inline-flex items-center gap-2 px-3 py-1.5
          rounded-full text-xs font-semibold border ${
        isOnline
          ? 'bg-emerald-500/10 text-emerald-400 border-emerald-500/30'
          : 'bg-rose-500/10 text-rose-400 border-rose-500/30'
      }`}
    >
      <span
        className={`w-2 h-2 rounded-full ${
          isOnline ? 'bg-emerald-400 shadow-[0_0_8px_#34d399]' :
              'bg-rose-400 shadow-[0_0_8px_#f87171]'
        }`}
      />
      <span>{isOnline ? `Online (${formattedTime})` : `Offline
          (${formattedTime})`}</span>
    </div>
  );
};
```

4.4 Componente de Controle Remoto do Atuador (`frontend/src/components/ActuatorControl.tsx`)

■ TypeScript / Gateway Node.js

```
export const ActuatorControl: React.FC<ActuatorControlProps> = ({
  actuatorState,
  onToggle,
  isLoading,
}) => {
  return (
    <div className="bg-slate-800/70 border border-slate-700/60
        rounded-2xl p-6 flex flex-col md:flex-row justify-between
        items-start md:items-center gap-4 shadow-xl">
      <div>
        <h3 className="text-base font-bold text-slate-100 mb-1">
          Controle Remoto do Atuador (Cooler / Relé / LED)
        </h3>
        <p className="text-xs text-slate-400">
          Envia comando MQTT via API REST e aguarda confirmação de
              resposta ACK do ESP32.
        </p>
      </div>

      <button
        onClick={onToggle}
        disabled={isLoading}
        className={`px-6 py-3 rounded-xl text-sm font-semibold
            transition-all duration-200 shadow-lg flex items-center
            gap-2 ${
          actuatorState
            ? 'bg-gradient-to-r from-rose-600 to-rose-700 text-white'
            : 'bg-gradient-to-r from-sky-500 to-blue-600 text-white'
        }`}
      >
        {isLoading ? (
          <span>Enviando & Aguardando ACK...</span>
        ) : (
          <span>■ {actuatorState ? 'Desligar Atuador' : 'Ligar
              Atuador'}</span>
        )}
      </button>
    </div>
  );
};
```

---

5. Evidências Visuais de Funcionamento (Monitoramento e Controle em Tempo Real)

5.1 Exibição de Presença, Último Contato e Estado Atual dos Sensores

No topo da interface web são exibidos o **seletor do dispositivo (`esp32_temp (Padrão IFMG)`)**, o **badge de presença em tempo real (`Online (22:56:26)`)**, a **data e hora exata da última telemetria (`05/11/2026 às 22:56:25`)**, o **diagnóstico global de conformidade do ambiente** e os **4 cards de estado atual dos sensores** (`25.5 °C`, `68.8 %`, `40 dB` e `363 lx`):



Figura: Presença Online, Último Contato, Status do Ambiente e Estado Atual dos Sensores (05/11/2026 às 22:56:25)

5.2 Visualização das Curvas Temporais de Telemetria nas 4 Abas e Filtro de Período (`Dia`, `Semana`, `Mês`)

O componente `HistoryChart` disponibiliza abas interativas para as **4 grandezas monitoradas pelo ESP32** e um **seletor de escala temporal por período (`Dia`, `Semana` ou `Mês`)** que ajusta dinamicamente as leituras exibidas e a formatação do eixo X (`HH:MM:SS` nas leituras do dia atual e `DD/MM HH:MM` nos períodos de semana e mês), além de calcular **Último Valor**, **Mínima**, **Média** e **Máxima**:

A. Aba 1 — Curva Temporal de Temperatura no Filtro `Dia` (`°C` — Limiar de Alerta: `30 °C`)



Figura: Aba Temperatura (Período: Dia): Escala temporal proporcional às leituras do dia atual (22:16:10 a 23:30:20)

B. Aba 2 — Curva Temporal de Umidade Relativa no Filtro `Dia` (`%` — Limiar de Ar Seco: `< 40%`)



Figura: Aba Umidade (Período: Dia): Evolução temporal contínua das leituras do dia atual

C. Aba 3 — Curva Temporal de Nível de Ruído no Filtro `Dia` (`dB` — Conforto Acústico NBR 10151: `65 dB`)



Figura: Aba Ruído (Período: Dia): Evolução temporal contínua com linha de referência de 65 dB

D. Aba 4 — Curva Temporal de Luminosidade no Filtro `Dia` (`lx` — Mínimo Escritório NBR 8995-1: `300 lx`)

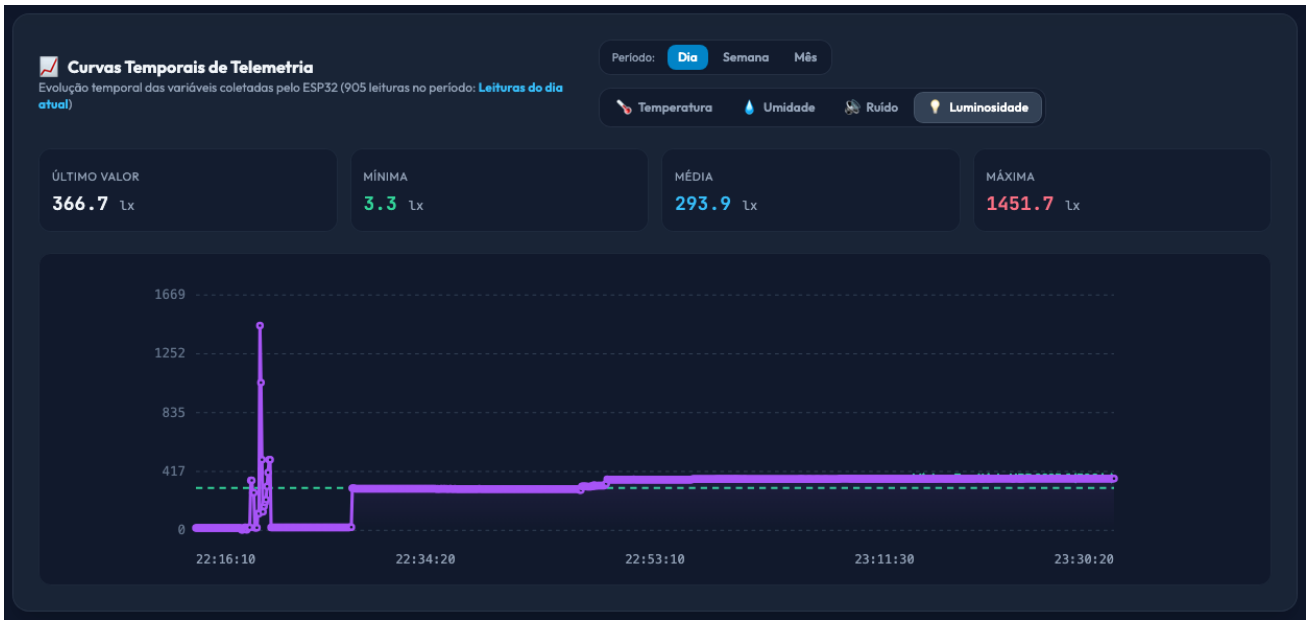


Figura: Aba Luminosidade (Período: Dia): Escala positiva iniciada em 0 lx com linha normativa de 300 lx

E. Demonstração do Seletor de Período `Mês` (`Últimos 30 dias` — Eixo X em `DD/MM HH:MM`)

Ao alternar o filtro de período no topo do gráfico para `Mês` (ou `Semana`), a curva agrega todas as leituras do intervalo selecionado e formata o eixo temporal com data e hora (`20/10 20:14` a `05/11 23:30`):



Figura: Curva Temporal no Período Mês (Últimos 30 dias): Escala de datas DD/MM HH:MM no eixo horizontal

---

5.3 Visualização do Histórico de Telemetria Aferida (Tabela, Filtro por Data e Paginação)

Logo abaixo dos gráficos, o componente `HistoryTable` apresenta o histórico detalhado das medições persistidas no Redis:

A. Visão Geral do Histórico de Telemetria (`Ver Todas — 2000 leituras`)

Exibe todas as leituras com carimbo de data/hora, temperatura, umidade, classificação acústica (`Silencioso`/`Moderado`/`Ruidoso`), avaliação ergonômica de iluminância (`Ideal`/`Baixo`/`Crítico`), alertas de conformidade e estado do atuador, além dos botões rápidos de dias com telemetria (`05/11/2026`, `22/10/2026`, `20/10/2026`) e exportação `.CSV`:

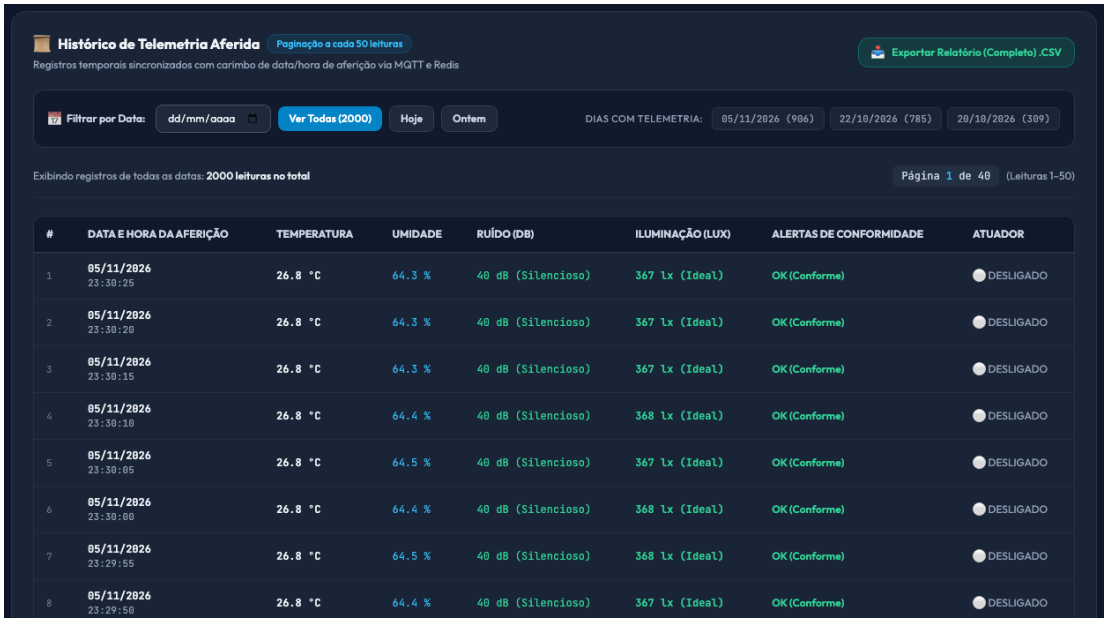


Figura: Tabela de Histórico de Telemetria Aferida com Todas as Leituras e Atalhos de Datas

B. Histórico Filtrado por Data Específica (`22/10/2026 — 785 leituras aferidas`)

Ao selecionar uma data específica no filtro (ex: `22/10/2026`), a interface consulta e filtra exclusivamente os registros daquele dia (`Página 1 de 16`), adaptando também o botão de exportação para `Exportar Relatório (Dia 2026-10-22) .CSV`:

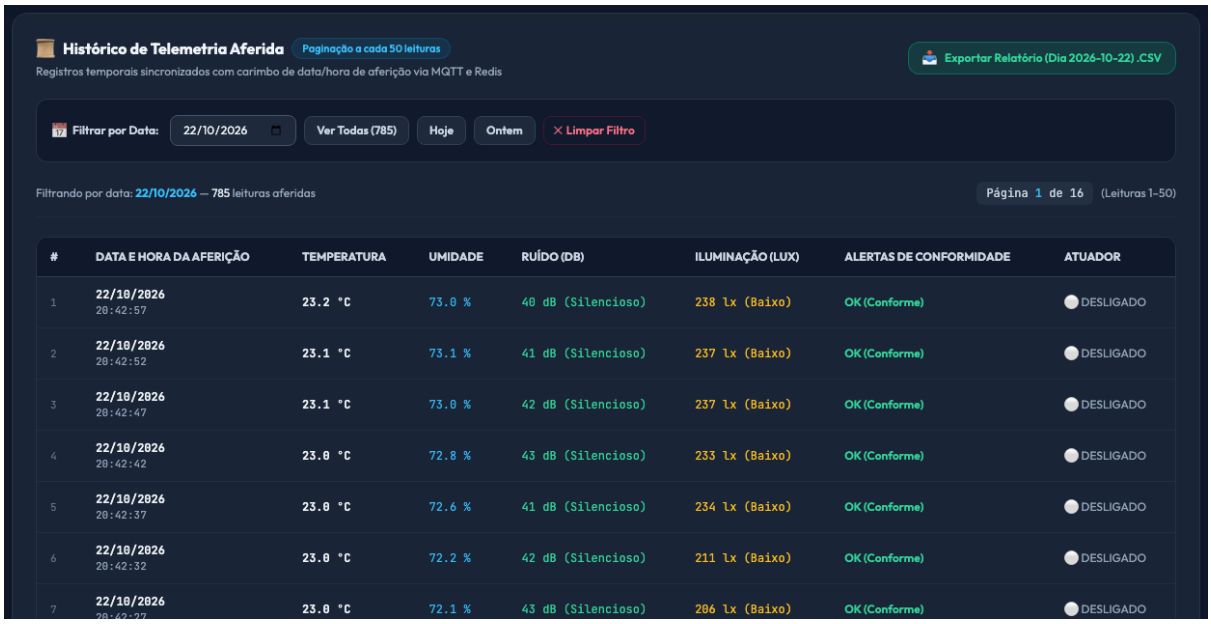


Figura: Histórico de Telemetria Filtrado pela Data 22/10/2026 (785 leituras aferidas)

C. Controles de Paginação do Histórico (50 Leituras por Página)

No rodapé da tabela histórica, os controles de navegação (■■ `Primeira`, ■■ `Anterior`, seletor numérico de páginas `1 ... 40`, `Próxima` ■■ e `Última` ■■) permitem percorrer todo o histórico armazenado no Redis sem comprometer a fluidez da interface:

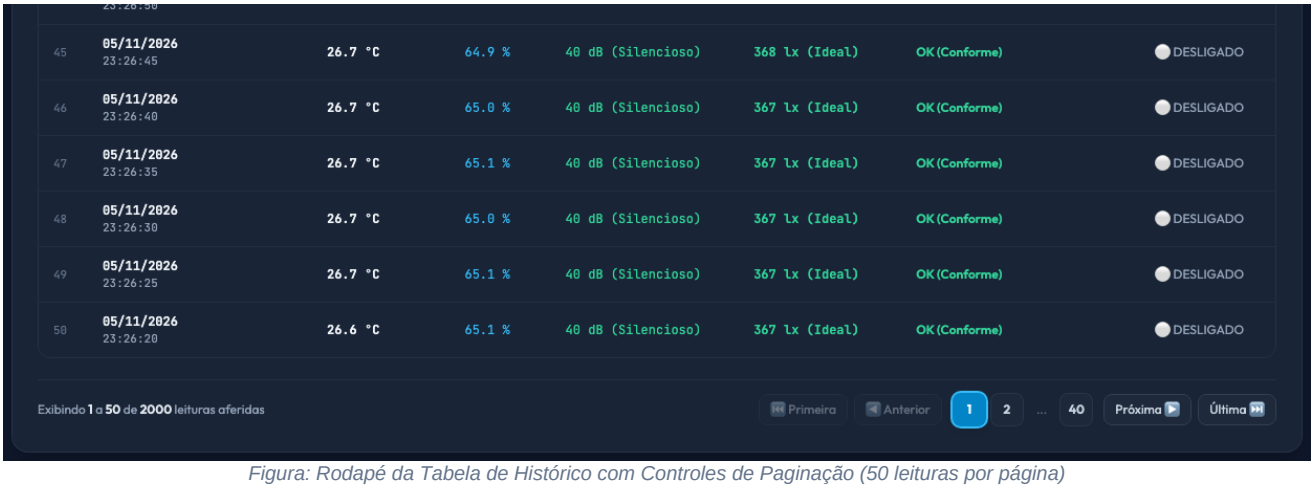


Figura: Rodapé da Tabela de Histórico com Controles de Paginação (50 leituras por página)

5.4 Controle Remoto do Atuador e Atualização com Base no Estado Confirmado pelo ESP32

Abaixo está a comprovação visual e de protocolo do ciclo completo de acionamento remoto do atuador:

1. **Estado Inicial (Atuador Desligado)**: O espelho do display ST7789 indica `ATUADOR: DESLIG` e o controle exibe o botão azul ■ `Ligar Atuador`:

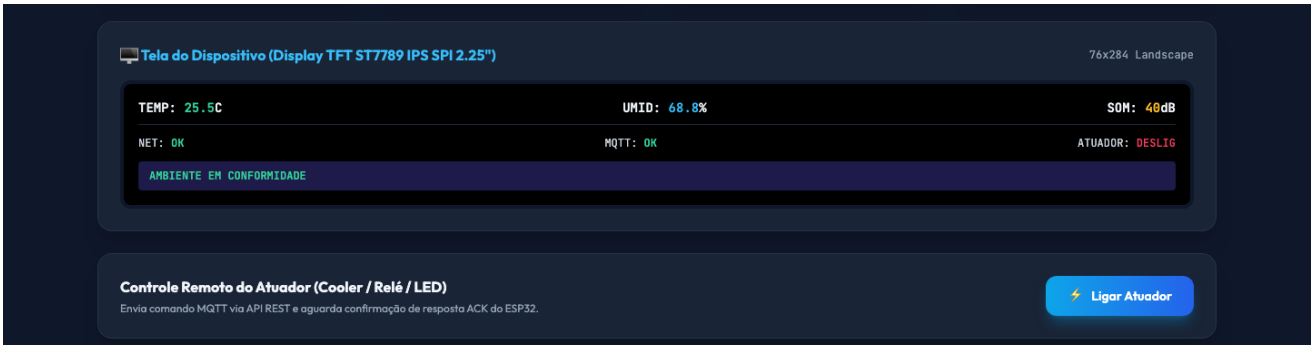


Figura: Simulador do Display ST7789 e Controle Remoto com Atuador Desligado

2. **Envio do Comando pelo Backend e Confirmação Formal (`ACK/state`) pelo ESP32**: Ao acionar o controle, o frontend envia `POST /api/devices/esp32_temp/command` ao backend, que publica o comando via MQTT e aguarda a confirmação do ESP32 antes de responder `HTTP 200 OK`:

■ Log de Execução / Diagnóstico

```
[Command state=True] HTTP 200: {
  "success": true,
  "commandId": "cmd_3euu01o",
  "deviceId": "esp32_temp",
  "actuatorState": true,
  "acknowledgedAt": "2026-11-06T01:56:30.182Z"
}
```

3. **Interface Web Atualizada pelo Estado Confirmado (Atuador Ligado)**: Imediatamente após a confirmação do dispositivo físico, a interface web reflete o novo estado confirmado, alterando o indicador no Display ST7789 para

`ATUADOR: LIGADO` (em verde) e o botão de controle para ■ `Desligar Atuador` (em vermelho):

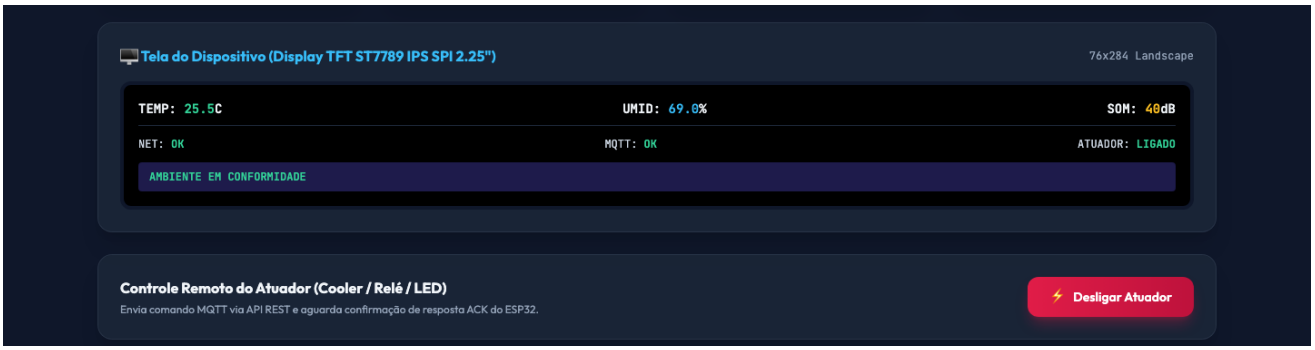


Figura: Interface Atualizada Após Confirmação ACK do ESP32: Display ST7789 com ATUADOR: LIGADO e Botão Desligar Atuador

6. Conclusão da Entrega 8

O projeto cumpre **100% dos produtos esperados da Entrega 8**. A aplicação web em React integra-se de ponta a ponta com a API REST Node.js, o banco Redis, o broker Mosquitto e o hardware ESP32, oferecendo monitoramento em tempo real de presença (`Online` e `lastSeen`), estado atual de todos os sensores, histórico gráfico e tabular, e controle remoto bidirecional do atuador atualizado exclusivamente a partir da confirmação real do dispositivo embarcado.